



ARQUITETURA DE SISTEMAS

AULA 3 - MODULARIZAÇÃO



INTRODUÇÃO À MODULARIZAÇÃO

- ❑ Módulo: Unidade independente que faz parte de um sistema maior, consistindo em um agrupamento lógico de código relacionado (por exemplo, um conjunto de classes ou funções com propósito comum).
- ❑ Modularidade: Princípio de organização arquitetural em que partes relacionadas do software são agrupadas em módulos coesos com interfaces bem definidas. Uma boa modularidade torna o sistema mais compreensível, facilitando a manutenção e evolução, mesmo sem requisitos explícitos para isso.
- ❑ Benefícios da modularização: Favorece reutilização de código, permite desenvolvimento e implantação isolada de funcionalidades e melhora a manutenibilidade.



EVOLUÇÃO HISTÓRICA DA MODULARIZAÇÃO

- ❑ Pré-OO e linguagens estruturadas: Nos anos 1960-70, surgem críticas ao código monolítico (ex: carta “Go To Considered Harmful” de Dijkstra). Isso impulsionou a programação estruturada e logo se percebeu a necessidade de agrupar logicamente partes do programa para melhor organização.
- ❑ Linguagens modulares: Linguagens como Modula e Ada introduziram explicitamente o construtor de módulo (sem classes) para agrupar código relacionado, precursor do que hoje conhecemos como pacotes/namespaces. Essa era modular pré-orientação a objetos foi breve, pois logo a OO ganhou popularidade.
- ❑ Orientação a objetos: A OO trouxe encapsulamento em classes, mas manteve o conceito de módulos para agrupamento lógico. Linguagens mantiveram mecanismos de modularização como pacotes em Java ou namespaces em .NET. Assim, módulos continuaram fundamentais, mesmo com novas abstrações.



EVOLUÇÃO HISTÓRICA DA MODULARIZAÇÃO

- ❑ Java 1.0 e namespaces: Os projetistas do Java criaram um sistema de pacotes alinhado ao sistema de arquivos para evitar conflitos de nomes de classes. Cada classe Java pertence a um pacote (ex: `com.empresa.produto`), correspondendo a pastas, garantindo nomes únicos: duas classes com o mesmo nome não convivem no mesmo pacote/diretório.
- ❑ Java 1.2 e JARs: Posteriormente, o Java introduziu os arquivos JAR (Java ARchive) para agrupar classes em um único arquivo. Isso facilitou a distribuição, mas reintroduziu riscos de conflito de nomes: dois JARs diferentes podem conter classes com o mesmo pacote/nome, levando a conflitos de carregamento de classes. Esse foi um dos motivadores para melhorias futuras na modularidade da plataforma (como o Java Module System no Java 9, embora não seja foco desta aula).



CONCEITOS-CHAVE DE MODULARIDADE

- ❑ **Coesão:** Mede o quão bem os componentes internos de um módulo estão relacionados entre si e trabalham para o mesmo propósito. Alta coesão significa que os elementos de um módulo fazem parte da mesma funcionalidade ou contexto; módulos coesos são desejáveis porque cada um “faz uma coisa bem feita”.
- ❑ **Acoplamento:** Mede o grau de dependência entre diferentes módulos. Representa quantos e quão fortes são os vínculos de um módulo com outros. Baixo acoplamento (módulos mais independentes) é desejável, pois mudanças em um módulo tendem a ter menos impacto nos demais.
- ❑ **Connascence:** Conceito avançado que qualifica dependências entre módulos. Duas partes estão em connascence se uma mudança em uma requer modificação na outra para manter o sistema correto. Fornece uma visão mais granular e qualitativa do acoplamento, classificando diferentes formas de dependência (nome, tipo, tempo, etc.).



COESÃO

- ❑ O que é coesão? É o grau em que os elementos dentro de um módulo estão relacionados e focados em uma única responsabilidade ou funcionalidade. Um módulo altamente coeso agrupa tudo que é essencial para desempenhar sua função específica, sem misturar coisas não relacionadas.
- ❑ Alta coesão é desejável: Quando um módulo tem coesão funcional elevada, todas as partes “pertencem” àquela unidade de software, facilitando compreensão e reutilização. Um módulo coeso tende a ser autocontido: remover ou separar suas partes quebraria sua função principal.
- ❑ Baixa coesão prejudica: Se um módulo reúne elementos que não têm relação forte, dizemos que há baixa coesão. Dividir um módulo já pouco coeso em partes menores não causa danos. Na verdade, pode melhorar a organização. Porém, dividir um módulo que era coeso artificialmente tende a aumentar o acoplamento e dificultar a manutenção. Ou seja, se forçar a separação de componentes que naturalmente deveriam estar juntos, você cria dependências desnecessárias entre módulos.



TIPOS DE COESÃO

- ❑ Coesão funcional: É a forma mais forte de coesão: todos os elementos do módulo contribuem diretamente para uma única função bem definida. Exemplo: um módulo de cálculo financeiro onde cada função (método) participa do processamento de uma mesma fórmula ou resultado final.
- ❑ Coesão sequencial: Ocorre quando a saída de uma parte do módulo é usada como entrada por outra parte, formando uma sequência de processamento dentro do módulo. Os componentes do módulo estão ligados em cadeia. Exemplo: função que lê dados, outra que processa esses dados e outra que gera um relatório a partir deles, tudo dentro do mesmo módulo.



TIPOS DE COESÃO

- ❑ Coesão comunicacional: Módulo onde suas partes operam sobre os mesmos dados ou contribuem juntas para alguma saída, mesmo que executem tarefas diferentes. Exemplo: em um método, primeiro salvar um registro no banco de dados e depois enviar um email usando esse registro: ambas operações distintas, mas comunicam-se via os mesmos dados.
- ❑ Coesão procedural: As partes do módulo devem ser executadas em uma ordem específica para funcionar corretamente. Há uma dependência de sequência temporal, mas pode não haver necessariamente compartilhamento significativo de dados entre elas. Exemplo: funções que implementam passos de um algoritmo complexo, que precisam ser chamadas numa ordem fixa.



TIPOS DE COESÃO

- ❑ Coesão temporal: Elementos agrupados apenas porque ocorrem próximos no tempo, por exigência de inicialização ou agendamento. Exemplo: uma rotina de startup do sistema que inicializa uma série de recursos diferentes. As ações não são relacionadas funcionalmente, mas são executadas juntas no momento de inicialização.
- ❑ Coesão lógica: Partes do módulo estão agrupadas mais por categoria ou semelhança lógica do que por funcionalidade específica. Exemplo: uma classe utilitária StringUtils com vários métodos estáticos para manipulação de strings. Eles tratam do mesmo tipo de dado (String), mas cada método realiza tarefas distintas e independentes (concatenar, buscar substring, converter caso etc.), ou seja, relacionados logicamente porém não funcionalmente interdependentes.
- ❑ Coesão coincidente (ou acidental): É a pior forma: os elementos de um módulo não possuem relação significativa, foram colocados juntos de forma arbitrária ou por coincidência. Indica falta total de planejamento na divisão em módulos. Exemplo: uma classe que reúna funções variadas sem conexão (calculadora de impostos junto com gerador de senhas, por exemplo).



MÉTRICA DE COESÃO - LCOM

- ❑ LCOM (Lack of Cohesion of Methods): Métrica quantitativa de coesão que indica falta de coesão em classes ou módulos. Em termos simples, o LCOM conta grupos de métodos que não compartilham variáveis/ atributos em comum. Quanto maior o valor, menos coeso o módulo está.
- ❑ Interpretação: Um LCOM alto sugere que a classe poderia ser dividida em classes menores, porque contém métodos/atributos desconectados entre si. Já um LCOM baixo (idealmente próximo de 0) indica alta coesão, ou seja, os métodos da classe tendem a usar os mesmos campos e colaborar uns com os outros.
- ❑ Exemplo prático: Considere uma classe Java ClientePedidos com dois campos: dadosCliente e listaPedidos. Se métodos como cadastrarCliente() e atualizarCliente() acessam apenas dadosCliente, enquanto métodos como criarPedido() e cancelarPedido() usam apenas listaPedidos, a classe apresenta baixa coesão. Seu LCOM seria alto, apontando que valeria separar funcionalidades de Cliente e Pedidos em módulos (classes) distintos.



ACOPLAMENTO

- ❑ O que é acoplamento? É o grau de dependência entre módulos de um sistema. Se alterações em um módulo impactam fortemente outro, eles estão fortemente acoplados. Buscamos sempre reduzir acoplamento, tornando módulos mais independentes (alto isolamento).
- ❑ Acoplamento Aferente (Ca): Métrica que conta quantos componentes externos dependem de um dado módulo (dependências de entrada). Ex.: uma classe utilitária chamada amplamente pelo sistema terá Ca alto, pois muitas outras partes “entram” nela.
- ❑ Acoplamento Eferente (Ce): Conta quantos componentes externos aquele módulo utiliza (dependências de saída). Ex.: um módulo de controle que chama muitos serviços externos ou classes de outros pacotes apresenta Ce alto, pois “sai” muito de sua fronteira.
- ❑ Baixo vs alto acoplamento: Baixo acoplamento significa Ca e Ce pequenos. O módulo não depende demais de outros nem é excessivamente dependido. Alto acoplamento significa muitas ligações de entrada/saída, o que tende a dificultar reutilização e aumenta o efeito cascata de mudanças.

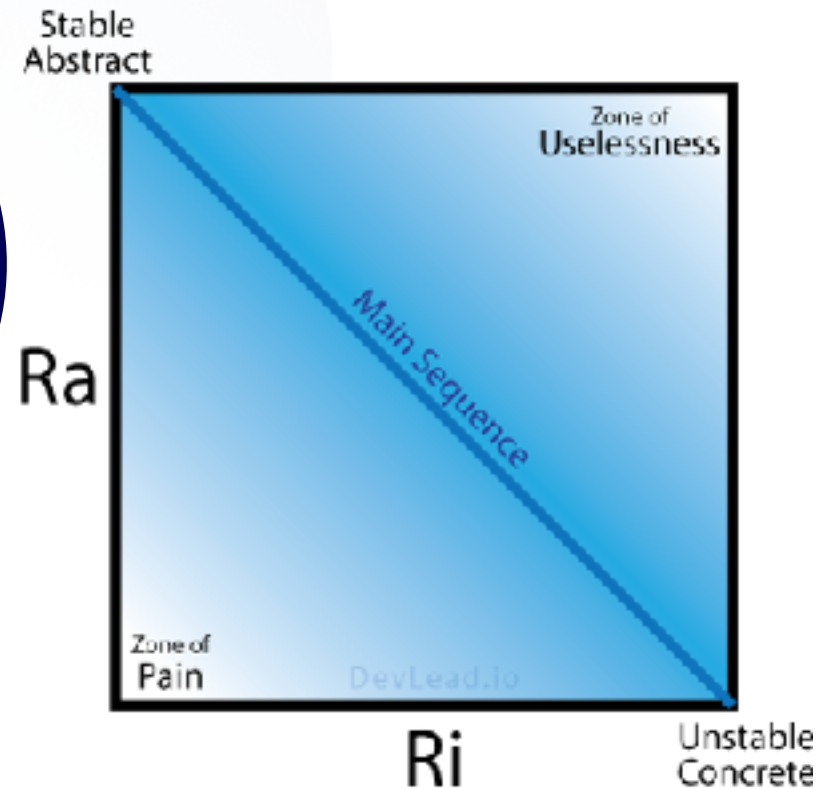


MÉTRICAS DE ACOPLAMENTO - ABSTRAÇÃO & INSTABILIDADE

- ❑ Abstractness (A): “Grau de abstração”: Métrica proposta por Robert Martin que representa a razão entre elementos abstratos (interfaces, classes abstratas) e elementos concretos em um módulo. A varia de 0 (nenhuma abstração, tudo concreto) a 1 (somente abstrações, nenhuma implementação). Um módulo altamente abstrato expõe principalmente interfaces/contratos e pouca implementação concreta.
- ❑ Instability (I): “Instabilidade”: Outra métrica de Martin, definida como a razão entre dependências de saída e o total de dependências (saída + entrada) do módulo. Formalmente $I = C_e / (C_e + C_a)$. I varia de 0 (completamente estável: ninguém depende de outros, só dependem dele) até 1 (completamente instável: depende de muitos outros e ninguém depende dele).
- ❑ Significado das métricas: Um módulo com instabilidade alta (próximo de 1) tende a quebrar facilmente se algo mudar, pois depende muito de outros componentes. Já um módulo com abstractness alto fornece oportunidades para extensão (boas abstrações), porém se for excessivamente abstrato sem ser usado pode virar “código morto”. O equilíbrio entre A e I é desejável.



EQUILÍBRIO ENTRE ABSTRAÇÃO E INSTABILIDADE (MAIN SEQUENCE)



Fonte: <https://devlead.io/DevTips/PrinciplesOfComponentCoupling#:~:text=Figure%20%20,Instability>

Gráfico Abstractness vs. Instability. A linha diagonal representa a Main Sequence (relação ideal $A + I = 1$). Componentes no canto inferior esquerdo (baixo A, baixo I) estão na “Zona de Dor”, e no canto superior direito (alto A, alto I) na “Zona de Inutilidade”

EQUILÍBRIO ENTRE ABSTRAÇÃO E INSTABILIDADE (MAIN SEQUENCE)

- ❑ Main Sequence: Idealmente, um módulo deve equilibrar seu grau de abstração e instabilidade. Existe uma relação ideal aproximada $A + I = 1$ (representada pela linha diagonal no gráfico), chamada de Main Sequence (sequência principal). Módulos próximos a essa linha têm mistura saudável de abstrações e dependências.
- ❑ Zone of Pain (Zona de Dor): Região no canto inferior esquerdo do diagrama. Módulos com baixa abstração (muito concretos) e baixíssima estabilidade (ou seja, instáveis, com muitas dependências externas). São implementações altamente acopladas a detalhes, sem interfaces flexíveis. Problema: difíceis de manter, qualquer mudança dói, pois estão fortemente dependentes e nada os protege (ex.: módulo monolítico cheio de chamadas ad-hoc a outros módulos).
- ❑ Zone of Uselessness (Zona de Inutilidade): Região no canto superior direito. Módulos com alta abstração (muitas interfaces, classes abstratas) mas muito estáveis (ninguém os utiliza, $I \sim 0$). Representa código potencialmente morto: muita abstração que não se reverte em uso prático. Problema: Desperdício de esforço e aumento desnecessário da complexidade arquitetural.
- ❑ Meta: Manter módulos próximos da Main Sequence. Se um componente cai nas zonas problemáticas, pode ser necessário refatorar. Por exemplo, adicionar abstrações a um componente na zona de dor para torná-lo mais flexível, ou consolidar/eliminar um componente na zona de inutilidade se ele for supérfluo.

CONNASCENCE

- ❑ Definição de Connascence: “Duas componentes são connascentes se uma mudança em uma requer uma mudança correspondente na outra para manter a correção do sistema. Ou seja, é um tipo de interdependência em que as partes praticamente “evoluem juntas”. Uma alteração sem a outra quebra o funcionamento.
- ❑ Origem do termo: Proposto por Meilir Page-Jones (1996) no artigo "What Every Programmer Should Know About Object-Oriented Design". Surge para dar nome e classificação a dependências mais sutis no código que vão além do acoplamento convencional.
- ❑ Dois tipos principais:
 - ❑ Connascence Estático (ou de compilação): visível no código fonte, relaciona componentes via estruturas estáticas (nomes, tipos, etc.)
 - ❑ Connascence Dinâmico: manifesta-se em tempo de execução (ordem de chamadas, tempo, valores correlacionados).



TIPOS DE CONNASCENCE ESTÁTICO (CÓDIGO)

- ❑ Connascence of Name (CoN): Múltiplos componentes dependem de um mesmo nome. Por exemplo, um método público chamado por diversos módulos. Se o nome do método mudar, todas chamadas devem mudar junto. Outro caso: duas classes acessam a mesma variável global `Config.URL`: se renomear essa variável, ambas quebram.
- ❑ Connascence of Type (CoT): Dependência de tipos de dados. Ocorre quando funções/módulos devem concordar sobre o tipo de uma entidade trocada entre eles. Por exemplo, um módulo envia uma `List<String>` esperando que o receptor trate exatamente desse tipo; se o tipo mudar para, digamos, `List<Objeto>`, o código interligado precisa mudar também.
- ❑ Connascence of Meaning/Convention (CoM/CoC): Dependência de significado ou convenção em valores. Exemplo clássico: usar valores numéricos com significado especial (ex.: `status = 0` para sucesso e `1` para erro). Todos os módulos que interpretam esses valores compartilham essa convenção; se você mudar o significado (digamos, introduzir `2` para outro status), todos os lugares que assumem a convenção antiga precisam mudar.



TIPOS DE CONNASCENCE DINÂMICO (EXECUÇÃO)

- ❑ Connascence of Execution (CoE): Ordem de execução importa entre componentes. Exemplo: em um sistema distribuído, o serviço B só pode rodar após o serviço A ter finalizado sua tarefa. Se a sequência for invertida ou concorrida fora de ordem, o resultado é incorreto.
- ❑ Connascence of Timing (CoT): (Aqui CoT refere-se a Timing) Momento ou temporização da execução importa. Exemplo: duas threads acessando um recurso compartilhado. Se a thread 2 executar antes da 1 completar uma operação crítica, ocorre uma race condition. Os componentes dependem do sincronismo correto no tempo.



TIPOS DE CONNASCENCE DINÂMICO (EXECUÇÃO)

- ❑ Connascence of Values (CoV): Múltiplos componentes dependem de valores correlacionados que devem mudar em conjunto. Exemplo: imagine dois objetos independentes que juntos representam um retângulo. Um armazena largura e outro altura. Se um muda o valor, o outro deve ser atualizado também para manter consistência (senão, representam retângulos diferentes). Ou campos duplicados em vários lugares que precisam ter o mesmo valor.
- ❑ Connascence of Identity (Col): Diversas partes do sistema devem referenciar a mesma entidade ou recurso. Exemplo: dois módulos usam a chave primária de um usuário para associar dados (ambos precisam se referir exatamente ao mesmo ID). Se um componente usa uma identificação diferente ou se perde a referência, a coesão entre eles se quebra. Ou duas variáveis que devem apontar para o mesmo objeto em memória para funcionar corretamente.



PROPRIEDADES DA CONNASCENCE

- ❑ **Força (Strength):** Medida de quão difícil é alterar ou refatorar uma forma de connascence. Formas “fortes” de connascence significam acoplamento mais rígido. Deve-se preferir connascences mais fracas, pois são mais fáceis de alterar. Regra geral: prefira dependências estáticas a dinâmicas (é mais simples descobrir e modificar referências de nome ou tipo no código do que sincronizar tempos de execução). Ex.: connascence de Nome é mais fraca (fácil renomear referência com IDE) do que connascence de Timing (difícil coordenar threads corretamente).
- ❑ **Localidade (Locality):** Refere-se à proximidade dos módulos connascentes. Connascence local (dentro de um mesmo módulo ou classe) é menos prejudicial do que connascence global (atravessando módulos distintos). Em outras palavras, se dois elementos fortemente dependentes estão encapsulados juntos, o impacto é menor. Conforme os componentes estão mais distantes (por exemplo, em serviços separados), deveríamos reduzir a força da connascence entre eles.
- ❑ **Grau (Degree):** Indica quantos componentes estão envolvidos na mesma connascence. Connascence de grau menor (poucas partes envolvidas) tende a ser menos danosa. Se uma forte dependência ocorre apenas entre duas classes, é mais fácil de gerenciar. Mas se a mesma convenção ou dependência amarra dezenas de classes, o impacto de mudança é muito maior. Sistemas grandes devem evitar connascences fortes de grau elevado, pois mudanças se tornam extremamente amplas.



REDUZINDO CONNASCENCE

- ❑ Diretrizes de Page-Jones:
 - ❑ (1) Minimize a connascence global, dividindo o sistema em partes encapsuladas onde for possível.
 - ❑ (2) Minimize connascence que atravessa fronteiras de encapsulamento. Se duas funcionalidades estão fortemente dependentes, considere colocá-las no mesmo módulo ou componente para isolá-las.
 - ❑ (3) Maximize a connascence dentro de cada encapsulamento. Ou seja, deixe as dependências fortes acontecerem apenas intramódulo, não entre módulos.



CONCLUSÃO

- ❑ Projeto modular eficaz:
 - ❑ Combine princípios de alta coesão e baixo acoplamento com a atenção às formas de conhecimento.
 - ❑ Ao projetar ou refatorar, identifique dependências ocultas e decida: posso eliminar ou reduzir essa dependência? Posso confiná-la dentro de um único módulo?
 - ❑ Assim, atingimos modularização que favorece evolução: cada módulo pode mudar com impacto mínimo nos demais, e quando há dependências inevitáveis, elas estão sob controle.



ATÉ A PRÓXIMA AULA